

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph), **Advanced** (how a state shape that every later step in this project keeps extending should actually be written). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A LangGraph graph is 3 things: a **state** shape (what data exists and flows through), **nodes** (plain functions that take the current state and return an update to it), and **edges** (which node runs after which). This step is the smallest possible version of all 3: one state field, two nodes that each touch it, one edge connecting them in a straight line, no branching, no conditional logic at all.

The whole point of this step is trust, not cleverness — prove to yourself that state actually flows from node to node the way you'd expect, by tracing it on paper first, then checking the code agrees.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The pieces to build:

```
# state.py
class GraphState(TypedDict):
    message: str

# nodes.py
def node_a(state: GraphState) -> dict:
def node_b(state: GraphState) -> dict:

# graph.py
def build_graph() -> CompiledGraph:
```

Each node function takes the whole state dict and returns a small dict of just the fields it's updating — LangGraph merges that dict into the running state for you, you don't hand back the whole thing rebuilt. `StateGraph(GraphState)` is where you register your state shape; `graph.add_node("a", node_a)` and `graph.add_node("b", node_b)` register the two functions; `graph.add_edge("a", "b")` connects them in order; `graph.set_entry_point("a")` (or the `START / END` sentinels, depending on your LangGraph version) tells it where to begin. `graph.compile()` turns the builder into something you can actually `.invoke({"message": "..."})`.

Sketch the state shape and both node signatures before writing any node body.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Nothing about this step's own job needs more than one field. But this exact `GraphState` is what Step 2 extends with a message history, Step 3 extends with found chunks, and Step 5 extends with an approval outcome — none of them replace it, per the README's own "What stays the same" notes across every step. That means the shape you pick right now, even for something this small, is a decision with a 5-step blast radius.

Two habits worth starting here rather than retrofitting later. First, give every field a real type, not `str` for everything just because it's convenient today — a `message: str` field that's actually always "the task the user asked" reads much better as `task: str`, because "message" gets genuinely ambiguous the moment Step 2 adds a real conversation history next to it. Second, trace by hand (as the README's own Step 1 instructions ask) not just what the state looks like after each node, but specifically *which node owns writing to which field* — in a 2-node graph this is trivial, but it's the exact discipline that keeps a 5-agent-worth-of-nodes graph (this project's eventual shape) readable, instead of every node reaching in and touching whatever it wants.

The extra pieces: - Name fields for what they'll mean by Step 5, not just what Step 1 needs — `task` instead of `message`, even though only Step 1 uses it so far. - A one-line comment on `GraphState` itself noting which node is expected to set each field, as a lightweight contract — cheap now, valuable once there are 5+ fields and several nodes.

Sketch the state shape with Step 5's eventual fields in mind (even though you won't write them yet) before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a correct, working 2-node graph — that's the entire ask of this step. Advanced treats the state shape as the one piece of this step's code that doesn't get thrown away later, and names it accordingly, because a rename mid-project (once 4 more nodes depend on the old name) is a much bigger deal than picking the better name now, for free.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state has one field: message (a string)

node_a(state): return {"message": state["message"] + " -> touched by A"}
node_b(state): return {"message": state["message"] + " -> touched by B"}

build_graph():
    create a StateGraph with the state shape
    add node_a, add node_b
    connect: entry -> a -> b -> end
    compile it

run it with {"message": "start"}, print the result
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
state.py:
    from typing import TypedDict

    class GraphState(TypedDict):
        message: str

nodes.py:
    from state import GraphState

    def node_a(state: GraphState) -> dict:
        return {"message": state["message"] + " -> touched by A"}

    def node_b(state: GraphState) -> dict:
        return {"message": state["message"] + " -> touched by B"}

graph.py:
    from langgraph.graph import StateGraph, START, END
    from state import GraphState
    from nodes import node_a, node_b

    def build_graph():
        builder = StateGraph(GraphState)
        builder.add_node("a", node_a)
        builder.add_node("b", node_b)
        builder.add_edge(START, "a")
        builder.add_edge("a", "b")
        builder.add_edge("b", END)
        return builder.compile()

main.py:
    graph = build_graph()
    result = graph.invoke({"message": "start"})
    print(result)
```

Run it, and by hand trace what `result["message"]` should be after each node before you check the printed output — that's the actual point of this step.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

state.py, field renamed for what it'll mean project-wide:

```
class GraphState(TypedDict):
    task: str # set by the caller before graph.invoke(); read by every later
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# state.py
from typing import TypedDict

class GraphState(TypedDict):
    task: str # the user's original request – set once, before the first node runs

# nodes.py
from state import GraphState

def node_a(state: GraphState) -> dict:
    """First touch: appends a marker showing node A ran."""
    return {"task": state["task"] + " -> touched by A"}

def node_b(state: GraphState) -> dict:
    # your turn: same idea as node_a, but mark it as node B
    ...
```

Fill in `node_b`, rename `graph.py`'s field references from `message` to `task`, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the graph mechanics (state, 2 nodes, 1 edge, compile, invoke) are identical across all 3 levels — this step genuinely doesn't need anything more complex than that. What changes is only the state field's name and the one-line comment on it, which is Advanced's whole point: the field this step defines is the one piece of code from Step 1 that survives, unchanged in shape, all the way to Step 5.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)